

Moses Dilts

PA2 Writeup

7-20-2026

Program Overview

As I am sure you are aware, the goal of this program is to detect collisions between arbitrary points and a given circle. I opted to use a spatial hash table to tackle this problem. The basic idea is to break down the search area (the entire 2d plane) into smaller units, “bins”, that are only searched if needed. This significantly reduces the amount of effort needed to determine if a point is inside the given area. The order of operations is as follows:

- Step 1: check to see if a file was given. If so, open it and move to step 2. If not, return EXIT_FAILURE.
- Step 2: determine the size of the hash bins and configure the plane structure (described in the ‘Structures Used’ section).
- Step 3: read the points from the file into the plane structure.
- Step 4: wait for a user input.
- Step 5: when a user input has been given, check all bins in a radius * 2 square centered on the circle radius.
- Step 6: calculate the distance from the center to all of the points in these bins. If they are colliding ($\leq$ circle radius), add them to the collided list.
- Step 7: return the list of collisions to the main function. If verbose mode is on (see the ‘Verbose Mode’ section), print the x,y coordinates. Otherwise, just print the total number of points.
- Step 8: repeat steps 4-7 until control+d is detected.
- Step 9: when control+d is detected, free the plane containing the loaded points along with all other memory and return EXIT_SUCCESS.
- Step 10 (optional): profit

(Custom) Structures Used

- plane: The plane structure represents a 2d plane filled with points. The structure

contains useful data such as the number of points held, the number of hash bins, and the width of the hash bins in space.

- point: (Do I really need to explain this one?) The plane structure holds the x and y coordinates of a 2d point.

Hash Overview

This program uses a spatial hash function cooked up by yours truly to group points by where they are on the 2d plane. The program will attempt to have each cell in the hash map contain the same (somewhat arbitrary) amount of points. As of the time of submission this was 250 points per bin. Each hash cell (I call them bins) maps a square area of space to a location in the table. The bin width is not fixed, it is instead calculated based on the target number of points per bin and the distribution of said points in space.

Verbose Mode

An optional argument “-v” can be used after the standard user input to activate “verbose mode.” When this is activated, the program will list both the number of collisions and the x,y coordinates of the colliding points.

Conclusion / Mo's Thoughts

Hashing is a very interesting topic to me, so I enjoyed implementing my own spatial hash. I went through several iterations, trying to minimize collisions as well as avoid things like integer overflow. The original function was modeled after a cryptographic hash (using xor), but I later moved to a more standard spatial hash that gives a hashed value related to the actual position of the input instead of a random output. In the future, I will probably experiment more with cryptographic hashing as I feel that this class has given me a good handle on the basics.